Laboratorium 5
Verilog – Układy Arytmetyczne

Instytut Telekomunikacji Multimedialnej

Materiały dydaktyczne przeznaczone są WYŁĄCZNIE dla studentów wydziału Elektronika i Telekomunikacja Politechniki Poznańskiej

Cele:

- zapoznanie z układami arytmetycznymi,
- realizacja mnożenia szerokiego za pomocą modułów o węższej szerokości bitowej.

Przebieg ćwiczenia:

1. Uruchomić program Mentor Modelsim, stworzyć nowy projekt, dodać pliki źródłowe.
2. Usunąć zawartość biblioteki „work”. Przeanalizować kod:

Plik testbench.v

```
module mytestbenchmodule();

reg CLK;
initial CLK  <= 0;
always #50  CLK <= ~CLK;

reg RST;
initial
begin
    RST <= 0;
    RST <= #100 1;
    RST <= #500 0;
end

reg [15:0] i_dat_a;
reg [15:0] i_dat_b;
reg i_stb;
reg o_ack;

initial
begin
    o_ack <= 0;
    i_stb <= 0;
    #1000;
    i_dat_a <= 15;
    i_dat_b <= 22;
    i_stb <= 1;
    #100;
    i_stb <= 0;
    #4050;
    o_ack <= 1;

end

adder
#(
.A_WIDTH(16),
.B_WIDTH(16)
)
adder1
(
.CLK(CLK),
.RST(RST),

.I_DAT_A(i_dat_a),
.I_DAT_B(i_dat_b),
.I_STB(i_stb),
.I_ACK(),

.O_DAT(),
.O_STB(),
```

```
    .O_ACK(o_ack)
    );

    endmodule
```

Plik adder.v

```
    module adder
    #(
    parameter A_WIDTH = 32,
    parameter B_WIDTH = 32
    )
    (
        input wire RST,
        input wire CLK,

        input  wire I_STB,
        output wire I_ACK,
        input  wire [A_WIDTH-1:0] I_DAT_A,
        input  wire [B_WIDTH-1:0] I_DAT_B,

        output reg  O_STB,
        output reg  [(A_WIDTH>B_WIDTH?A_WIDTH:B_WIDTH) : 0] O_DAT,
        input  wire O_ACK
    );

    assign I_ACK = I_STB & ~O_STB;

    always @(posedge CLK or posedge RST)
    if (RST) O_DAT <= 0; else if (I_ACK) O_DAT <= I_DAT_A+I_DAT_B;

    always @(posedge CLK or posedge RST)
    if (RST) O_STB <= 0; else if (O_ACK) O_STB <= 0; else if (I_ACK) O_STB <= 1;

    endmodule
```

3. Co oznaczają parametry A_WIDTH i B_WIDTH? Jakie zalety ma ich zdefiniowanie?
4. Zastanowić się, z czego wynika zapis:

```
output reg  [(A_WIDTH>B_WIDTH?A_WIDTH:B_WIDTH) : 0] O_DAT,
```

5. Skompilować i uruchomić symulację. Wykreślić przebiegi czasowe modułu adder1.
6. Czy układ działa poprawnie? Sprawdzić z innymi wartościami liczb. Sprawdzić inne sposoby podawania danych (kilka liczb w jednym testbench'u itp). Zamieścić zmodyfikowany TB.
7. Stworzyć nowy program multiplier.v modyfikując odpowiednio kopię pliku adder.v. Program ten ma realizować funkcję mnożenia.

Plik multiplier.v

```
    module multiplier
    #(
    parameter A_WIDTH = 32,
    parameter B_WIDTH = 32
    )
    (
        input wire RST,
        input wire CLK,

        input  wire I_STB,
        output wire I_ACK,
        input  wire [A_WIDTH-1:0] I_DAT_A,
        input  wire [B_WIDTH-1:0] I_DAT_B,

        output reg  O_STB,
        output reg  [xxxxxxxxxxxxxxxxx:0] O_DAT,
        input  wire O_ACK
    );

    ....
```

8. Co powinno się znaleźć w miejscu symbolu xxxxxxxxxxxxxxxxx aby układ działał poprawnie?
9. Poprawić testbench, aby testował moduł multiplier dla szerokości bitowej 16 danych wejściowych.
10. Skompilować i uruchomić symulację. Wykreślić przebiegi czasowe modułu multiplier.

11. Czy układ działa poprawnie? Sprawdzić z innymi wartościami liczb poddawanych konwersji. Sprawdzić inne sposoby podawania danych (kilka liczb w jednym testbench'u itp) podobnie jak w punkcie 6.

12. Napisać nowy program realizujący funkcję mnożenia 32-bitowego z wykorzystaniem podrzędnych modułów mnozenia 16-bitowego i dodawania. Program powinien zaczynać się następująco:

Plik multiplier_32bit.v

```
module multiplier_32bit
(
    input wire RST,
    input wire CLK,

    input  wire I_STB,
    output wire I_ACK,
    input  wire [31:0] I_DAT_A,
    input  wire [31:0] I_DAT_B,

    output wire  O_STB,
    output wire [63:0] O_DAT,
    input  wire O_ACK
);

....
```

Zasada mnożenia jest jak przy mnożeniu "pisemnym":

Liczba A				A_H	A_L
Liczba B	*			B_H	B_L
					B_L* A_L
	+		B_L* A_H		
	+		B_H * A_L		
	+	B_H * A_H			

13. Skompilować i uruchomić symulację. Wykreślić przebiegi czasowe modułu multiplier_32bit.
14. Czy układ działa poprawnie? Sprawdzić z innymi wartościami liczb poddawanych konwersji. Sprawdzić inne sposoby podawania danych (kilka liczb w jednym testbench'u itp) podobnie jak w punkcie 6.
15. Uruchomić środowisko Lattice Diamond - porównać wyniki syntezy dwóch implementacji mnożenia 32-bitowego (multiplier_32bit.v oraz multiplier.v modyfikując parametry).
16. Utworzy nowy projekt (File -> New Project).
    Nadać nazwę np. „mul_test”, utworzyć pusty projekt bez plików wejściowych.
    Wybrać układ który będzie wykorzystywany na zajęciach MachXO2-4000HC 4TG144C
    Rekomendowane narzędzie syntezy to Synplify Pro.
17. Dodać nowy plik projektu (Zakładka File List -> prawym na Input Files -> Add -> New File)
    Zwyczajowa nazwa głównego pliku to TOP.v
    Przekopiować zawartość potrzebnych plików źródłowych do pliku TOP (bez mytestbench!) lub dodać kolejne pliki źródłowe.
18. W zakładce Process poprosić o syntezę projektu i wygenerowanie pliku JDEC.
    W takiej konfiguracji wszystkie wejściowe porty głównego modułu (powinien to być TOP.v co w zakładce File List informuje pogrubienie – jeśli tak nie jest to prawym na TOP.v -> Set Top-Level Unit) zostaną losowo przypisane do nóżek FPGA (plik .lpf zawiera informacje o dokładnych preferencjach.. dla sprawdzenia rozmiaru syntezowanego układu nie jest to bardzo istotne).
19. Wyniki syntezy można przeglądać w zakładce Output lub jeśli synteza przebiegła poprawnie w zakładce Reports (polecane pozycje to Synplify Pro oraz Map)
20. Odczytać maksymalną częstotliwość oraz wyniki syntezy HDL – ilość zajętych komórek logicznych. Która z implementacji okazała się wydajniejsza? Dlaczego?